

Resumen integral y unificado de diseño de componentes de software, convención, diseño interactivo para front-end y back-end

Una de las arquitecturas mencionadas es el patrón **MVC** para el diseño de backend, su filosofía consta de organizar por responsabilidades a distintos aspectos del código, tales como las rutas (**define qué URL activa qué controlador sin ejecutar lógica propia**), controladores (**Recibe las peticiones (req), coordina la lógica llamando al modelo y devuelve la respuesta (res)**), modelos (**la preparación de información e interacción con la BD son exclusivamente manejadas por estos**), y vistas (**respuesta que llega al cliente, ya sea como JSON en APIs o HTML mediante plantillas como EJS**).

En cuanto al front-end, vimos el marco para construir sistemas que escalables en complejidad, llamado **Atomic Design**. Consta de las siguientes entidades, Átomos y Moléculas (**representan elementos básicos con la mínima funcionalidad, reutilizables**), Organismos y Plantillas (**Secciones complejas con contexto (navbar, grids) que definen el layout de la página**), Jerarquía Visual (**El diseño debe guiar el ojo mediante el uso intencionado de tamaño, contraste, color y espacio en blanco**)

En términos de User Experience y Diseño interactivo, analizamos la importancia de reducir el "gap" entre el modelo mental del usuario y el funcionamiento real del sistema, usando principios como los siguientes:

- Los elementos deben comunicar visualmente cómo se usan y el sistema debe responder de inmediato a cada acción para evitar confusión.
- **Es obligatorio diseñar no solo el estado "normal", sino también los estados de Carga, Error, Éxito, Vacío y Deshabilitado.**
- Se deben aplicar los principios de visibilidad del estado del sistema y la prevención de errores.

Y finalmente, repasamos reglas para garantizar la calidad y protección de los datos, como las siguientes prácticas:

- Nunca se debe concatenar input del usuario en strings SQL, sino que deben usarse **consultas parametrizadas** (\$1, \$2) para evitar inyecciones.
- **Las variables de entorno**, ([p.ej](#) las de `.env`) nunca se suben al repositorio.
- Las interfaces deben ser **POUR**: Perceptibles, Operables, Comprensibles y Robustas.
- Se deben etiquetar los commits asistidos como `[AI-assisted]` y proporcionar una justificación técnica que conecte la solución con el criterio de ingeniería.

Convención de código del equipo

Guia de estilo para desarrollo web

Criterio	Descripción	Ejemplos de uso	
		Válido	Erróneo
Variables	Denominación de variables que exprese intención, utilizando camelCase (evitando abreviaturas o nombres genéricos).	nombreUsuario	x, dato, temp
Strings	Escritas entre comillas “”, de formar parte de la interfaz deben mantener una redacción consistente y concisa; la inserción de variables será a través de <i>literals</i> con <i>backticks</i> .	nombreVariable = ‘Hola, \${nombreUsario}’;	-
Funciones	Denominación de funciones que denote una acción a través de un verbo, con formato camelCase. Si realiza varias tareas,	function verboNombre() validarUsuario()	function datos()

Criterio	Descripción	Ejemplos de uso	
	valorar el uso de funciones auxiliares.		
Parámetros	Sólo declarar aquellos que sean necesarios. Denominaciones descriptivas. De requerir varios parámetros de una entidad, empaquetar en un objeto.	function validarUsuario (datosUsuario) {}	function validarUsuario (nombre, correo, edad, dirección, telefono) {}
Comentarios	Usarlos para aportar contexto, explicar procesos complejos, aclarar decisiones de diseño, o marcar pendientes. Evitar uso de /* ... */	// Este es un comentario. // Hola mundo.	/* Este es un comentario. Hola mundo. */
Objetos	Deben tener un nombre descriptivo y mantener una estructura clara, siguiendo la convención camelCase y manteniendo un orden lógico en sus atributos.	const perfilUsuario = { primerNombre: 'Víctor', apellidoPaterno: 'García', apellidoMaterno: 'Galván' };	-
Estatutos de Control	Deben escribirse con indentación consistente y siempre se utilizarán llaves incluso si solo cuenta con una línea. Se prioriza la legibilidad, no la reducción de líneas.	if (usuarioValido) { despliegaDashboard(); } else { redirigeLogin(); }	-

Guía de estilo para queries y base de datos

Criterio	Descripción	Ejemplos de uso
Variables	Deben tener nombres descriptivos y usar un	p_id_usuario v_fecha_inicio

Criterio	Descripción	Ejemplos de uso
	prefijo que identifique a la variable, como p_ para parámetros, seguido del nombre en snake_case.	
Strings	Las cadenas de texto se escribirán preferentemente con comillas simples (' '), evitando concatenaciones innecesarias.	WHERE estado = 'activo'
Funciones	Las funciones deben iniciar con un verbo, representando una acción y escrita en snake_case.	actualizar_expediente registrar_usuario
Parámetros	Deben contener nombres descriptivos, evitando aquellos que sean innecesarios y deben iniciar con un prefijo para distinguir de columnas o variables locales.	CREATE PROCEDURE obtener_cliente (p_id_cliente INT)
Comentarios	Utilizar para explicar consultas complejas o reglas de negocio. Pueden ser de una sola línea o de bloque según sea necesario. Evitar redundancia.	-- Hola soy un comentario de una linea /* Soy un comentario de varias líneas */
Objetos	Nombres descriptivos, consistentes y en snake_case, determinar denominación singular o plural para todo el proyecto. No usar espacios ni caracteres especiales.	Tablas: clientes, usuarios, contratos. Columnas: id_cliente, nombre_usuario. Vistas: prefijo vw_ Triggers: prefijo trg_
Estatutos de Control	Las palabras reservadas deben escribirse en mayúsculas y deben estar claramente indentadas. Se evitarán bloques excesivamente anidados y cada bloque deberá estar	IF v_total_pagos > 0 THEN UPDATE contratos SET estado = 'ACTIVO' WHERE id_contrato = p_id_contrato; ELSE UPDATE contratos SET estado = 'PENDIENTE'

Criterio	Descripción	Ejemplos de uso
	bien delimitado y organizado.	WHERE id_contrato = p_id_contrato; END IF;

Estrategia de ramas

- **main**: rama principal estable
- **develop**: rama de integración de cambios
- **features**: ramas temporales para tareas específicas

Formato de ramas:

Ejemplos:

- victor/avance1
- luis/avance2

Formato de commits

Tipos:

- feat: nueva funcionalidad
- fix: corrección de errores
- docs: documentación
- refactor: reorganización del código
- test: pruebas
- chore: mantenimiento

Ejemplos:

- feat: agrega página principal
- fix: corrige error en formulario
- docs: actualiza README

Contribuciones individuales al front y back-end

Estuve a cargo del diseño conceptual de las siguientes interfaces:

- Alertas Globales
- Operaciones Globales
- Clientes
- Alta de Cliente
- Alta de Contrato
- Alta de Operación
- Listas de Bloqueo

- Subir Documentos

Y estuve a cargo de la implementación de la subida de listas de bloqueo, parametrización de ebr, creación de reportes ROIP, diseño y corrección de la base de datos actual, así como del despliegue en vercel.